

Day 15 — Python for Security Engineering: Foundations

Name: Mohammed Farhan
Role: Associate – Information Security (Intern)
Organization: CyberShelter UAE
Date: 18 July 2026

Objective

Build foundational Python skills for security automation — file handling, exception handling, command-line argument parsing, and network lookups — by writing a working IP-checking script. This forms the base that will be extended into a full IOC enrichment tool on Day 17.

Hands-On Task

Wrote a Python script (`ip_checker.py`) that:

1. Accepts a text file of IP addresses via a command-line argument
2. Performs a reverse DNS lookup on each IP using Python's socket library
3. Checks each IP against a hardcoded blocklist (simulating a threat intelligence feed)
4. Outputs the results as a formatted table using the rich library, with columns: IP, Reverse DNS, In Blocklist, Risk Level

Risk Level logic:

IPs found in the blocklist are rated High; IPs with no reverse DNS record are rated Medium; all others are rated Low.

Key Concepts Applied

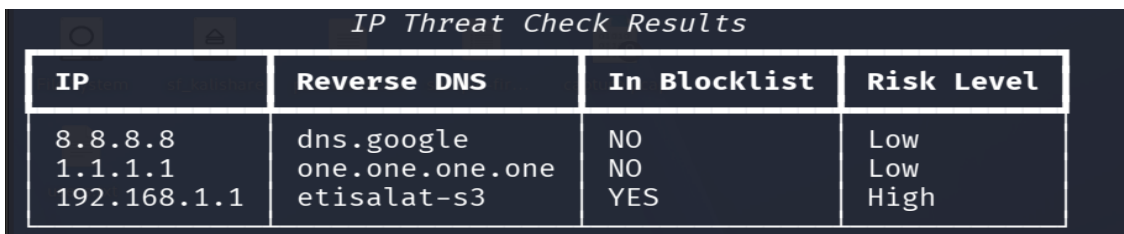
- **File I/O** — reading a text file line by line, wrapped in exception handling for a missing file
- **Exception handling (try/except)** — used at three separate points: invalid IP format, DNS lookup timeout/no-record, and missing input file, ensuring the script processes the full list without crashing on any single bad entry
- **argparse** — accepts the input filename via a `-f / --file` command-line flag, avoiding the need to hardcode filenames
- **socket library** — used `socket.gethostbyaddr()` for reverse DNS lookups and `socket.setdefaulttimeout()` to prevent the script from hanging indefinitely on an unresponsive DNS server
- **rich library** — used to render clean, aligned terminal output instead of manually formatted print statements

Development and Testing

The script was built incrementally: file reading and argument parsing first, then reverse DNS lookup, then blocklist checking, and finally the rich table output and edge-case handling — each stage tested independently before adding the next.

Test 1 — Normal input:

- `python3 ip_checker.py -f test_ips.txt`

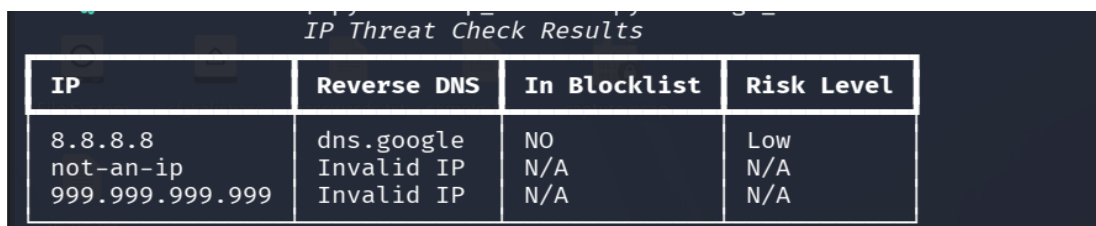


IP	Reverse DNS	In Blocklist	Risk Level
8.8.8.8	dns.google	NO	Low
1.1.1.1	one.one.one.one	NO	Low
192.168.1.1	etisalat-s3	YES	High

Confirmed correct output for 3 real IPs, including one deliberately placed in the blocklist to verify detection logic.

Test 2 — Edge cases (invalid IP, malformed IP):

- `python3 ip_checker.py -f edge_test.txt`



IP	Reverse DNS	In Blocklist	Risk Level
8.8.8.8	dns.google	NO	Low
not-an-ip	Invalid IP	N/A	N/A
999.999.999.999	Invalid IP	N/A	N/A

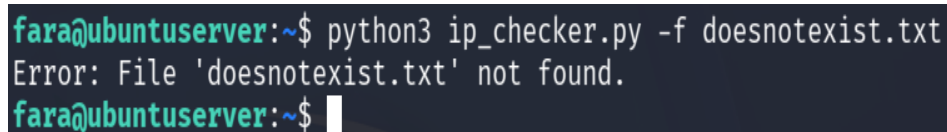
Confirmed the script correctly labeled invalid entries as "Invalid IP" and continued processing remaining valid entries rather than crashing.

Test 3 — Missing file:

bash

- `python3 ip_checker.py -f doesnotexist.txt`

Confirmed the script displayed a clear error message and exited gracefully, rather than showing a raw Python error/traceback.



```
fara@ubuntu:~$ python3 ip_checker.py -f doesnotexist.txt
Error: File 'doesnotexist.txt' not found.
fara@ubuntu:~$
```

Deliverable

- `ip_checker.py` — working script, committed to Git repo (week-4/day-15-deliverables/)
- `README.md` — usage instructions
- All 3 required edge cases (invalid IPs, DNS timeouts, file not found) confirmed handled without crashing

Conclusion

Today established the core Python building blocks — file handling, exception handling, command-line arguments, and network lookups — that every subsequent security automation script this week will build on. The script was deliberately built incrementally and tested at each stage rather than written all at once, confirming each piece worked before adding complexity. This same `ip_checker.py` script will be directly extended on Day 17 into a full IOC enrichment tool, adding VirusTotal, AbuseIPDB, and Shodan API lookups on top of the reverse DNS and blocklist logic already working today.